Construcción de la Base de Datos Staging a partir de la Base de Datos Jardinería: Análisis, Diseño ETL y Documentación Técnica

Equipo de Trabajo:

Carlos Andres Osorio Agudelo

Estefania Martínez Guzman

Facultad de Ingeniería y Ciencias Agropecuarias

IU Digital de Antioquia

Bases de Datos 2

Antonio J. Valderrama J.

Marzo 27 de 2026

Resumen

El presente documento expone el proceso de análisis, diseño e implementación de una base de datos de tipo Staging, construida a partir de la base de datos operacional denominada Jardinería, administrada en SQL Server. El trabajo comprende una auditoría campo por campo de las tablas origen, la identificación de los datos con valor analítico y la eliminación de los campos operacionales irrelevantes para el análisis de ventas. Como resultado del análisis se diseñó un modelo de seis tablas desnormalizadas que reduce de ocho a seis las entidades del modelo, fusionando información de tablas relacionadas mediante columnas calculadas y persistidas. Se documentan las consultas DDL para la creación de la estructura Staging, las sentencias ETL de extracción, transformación y carga, los procedimientos de validación de integridad y los comandos de respaldo de ambas bases de datos. El enfoque adoptado permite ejecutar consultas analíticas complejas con un número mínimo de JOINs, mejorando significativamente el rendimiento y la legibilidad de las consultas de inteligencia de negocios.

Palabras clave: base de datos staging, ETL, desnormalización, SQL Server, Jardinería, análisis de ventas, backup.

Construcción de la Base de Datos Staging a partir de la Base de Datos Jardinería: Análisis, Diseño ETL y Documentación Técnica

Introducción

En el contexto actual de la gestión de datos empresariales, las organizaciones requieren soluciones que permitan transformar datos operacionales en información lista para el análisis estratégico. Las bases de datos de staging constituyen una capa intermedia fundamental en los procesos de Extracción, Transformación y Carga (ETL), cuya función principal es depurar, consolidar y estructurar los datos provenientes de sistemas transaccionales antes de ser utilizados en herramientas de inteligencia de negocios.

La base de datos Jardinería, operada sobre SQL Server, contiene la información comercial de una empresa distribuidora de productos de jardinería: empleados, clientes, productos, pedidos, detalles de pedido y pagos. Esta base de datos, diseñada bajo principios de la Tercera Forma Normal (3FN), incluye campos de carácter operacional —como extensiones telefónicas, rutas de imágenes, direcciones físicas y campos HTML— que no aportan valor al análisis de ventas pero incrementan la complejidad de las consultas.

El presente trabajo documenta el proceso de construcción de la base de datos Staging, la cual aplica dos principios fundamentales: la eliminación agresiva de campos operacionales y la desnormalización completa del modelo, pasando de ocho tablas a cinco entidades más la tabla de pagos, lo que reduce drásticamente el número de JOINs necesarios en las consultas analíticas.

Objetivos

Objetivo General

Diseñar e implementar una base de datos Staging optimizada para el análisis de ventas, a partir de la base de datos operacional Jardinería, aplicando técnicas de limpieza, desnormalización y generación de columnas calculadas en SQL Server.

Objetivos Específicos

- Auditar campo por campo las tablas de la base de datos Jardinería, clasificando cada atributo según su relevancia para el análisis de ventas.
- Diseñar la estructura DDL de las cinco tablas que conforman la base de datos Staging, incluyendo columnas desnormalizadas y calculadas persistidas.
- Construir y ejecutar las consultas ETL que extraigan, limpien y carguen los registros desde Jardinería hacia la base de datos Staging.
- Implementar procedimientos de validación que garanticen la integridad, completitud y coherencia de los datos trasladados al Staging.
- Generar los archivos de respaldo (backup) de ambas bases de datos y documentar los procedimientos de restauración correspondientes.

Planteamiento del Problema

La base de datos Jardinería ha sido diseñada bajo un enfoque transaccional, priorizando la integridad referencial y la eliminación de redundancias mediante la normalización. Si bien este diseño es adecuado para las operaciones del día a día, presenta limitaciones significativas cuando se requiere realizar análisis de ventas, generación de reportes o consultas de inteligencia de negocios.

Entre las principales dificultades identificadas se encuentran: la necesidad de ejecutar múltiples JOINs entre tablas para obtener información básica de negocio (hasta cinco JOINs para identificar la oficina con mayores ventas), la presencia de campos operacionales sin valor analítico que incrementan el volumen de datos procesados —como números de fax, extensiones telefónicas, descripciones HTML y rutas de imágenes—, y la ausencia de columnas calculadas que permitan agilizar métricas recurrentes como el margen bruto, el importe por línea de pedido o el análisis temporal por año y mes.

Adicionalmente, la ausencia de una capa Staging correctamente estructurada implica que cada consulta analítica debe reconstruir la misma lógica de unión y transformación, generando código repetitivo, propenso a errores y de difícil mantenimiento. Esta situación justifica la construcción de una base de datos Staging que opere como capa intermedia optimizada para el consumo analítico.

Análisis del Problema

El análisis del problema se realizó mediante una auditoría exhaustiva de los datos almacenados en la base de datos Jardinería. Para cada tabla origen se evaluó cada campo según tres criterios: si el campo aporta valor al análisis de ventas, si es un dato operacional o logístico sin relevancia analítica, y si puede ser desnormalizado en otra tabla para reducir la necesidad de JOINs. Este proceso constituye la base del diseño del modelo Staging.

Estructura Original de la Base de Datos Jardinería

La base de datos Jardinería está compuesta por ocho tablas: oficina, empleado, cliente, producto, Categoria_producto, pedido, detalle_pedido y pago. Esta estructura sigue los principios de la Tercera Forma Normal, lo que significa que la información está distribuida entre múltiples tablas relacionadas mediante claves foráneas.

Resultados de la Auditoría de Campos

El proceso de auditoría arrojó los siguientes resultados globales sobre los 65 campos evaluados en las ocho tablas de la base de datos origen:

- 22 campos fueron marcados como CONSERVA: son atributos con valor directo para el análisis de ventas, como identificadores, nombres comerciales, fechas, estados, montos y cantidades.

- 25 campos fueron marcados como DESCARTA: corresponden a datos de contacto operacional (teléfonos, fax, extensiones), direcciones físicas, campos HTML, imágenes y jerarquías internas que no aportan al análisis.
- 11 campos fueron marcados como DESNORM: atributos de tablas secundarias que se incorporan directamente en las tablas principales del Staging para evitar JOINs repetitivos.
- 7 campos son de tipo CALCULADO: columnas derivadas de operaciones sobre datos existentes, como margen bruto, importe por línea, año y mes de pedido, y días de retraso.

Decisiones de Diseño Principales

Como resultado del análisis, se tomaron tres decisiones de diseño estructurales que definen el modelo Staging :

Primera decisión — Eliminación de STG_OFICINA: la tabla oficina desaparece como entidad independiente en el Staging. Sus campos analíticamente relevantes (código, ciudad, país y región de oficina) se desnormalizan directamente en STG_EMPLEADO, eliminando así una tabla completa y sus correspondientes JOINs.

Segunda decisión — Eliminación de STG_CATEGORIA_PRODUCTO: de manera análoga, la tabla Categoria_producto se disuelve en el Staging. El único campo de valor analítico, el nombre de categoría, se desnormaliza en STG_PRODUCTO y adicionalmente en STG_DETALLE_PEDIDO, permitiendo consultas por categoría sin necesidad de ningún JOIN adicional.

Tercera decisión — Desnormalización en cascada del representante de ventas: el nombre completo, puesto y datos de oficina del representante de ventas se incorporan directamente en STG_CLIENTE, evitando el JOIN recurrente hacia STG_EMPLEADO en las consultas de desempeño comercial.

Propuesta de la Solución

Descripción del Análisis Realizado a los Datos de Jardinería y su Traslado al Staging

Diseño de la Estructura del Staging

La base de datos Staging se estructura en seis tablas, creadas en el siguiente orden para respetar las dependencias de claves foráneas: STG_EMPLEADO, STG_CLIENTE, STG_PRODUCTO, STG_PEDIDO, STG_DETALLE_PEDIDO y STG_PAGO. A continuación, se describe cada tabla y sus características principales.

STG_EMPLEADO: absorbe la tabla oficina. Contiene el identificador del empleado, el nombre completo consolidado (nombre + apellido1 + apellido2), el puesto, y los cuatro campos de oficina desnormalizados: código, ciudad, país y región. Los datos de contacto (extensión, email), la FK de oficina y la jerarquía de jefe son descartados.

STG_CLIENTE: absorbe los datos del representante de ventas y su oficina. Conserva nombre del cliente, ciudad, región, país y límite de crédito. Incorpora cuatro columnas desnormalizadas del

representante: nombre completo, puesto, ciudad y país de la oficina. Se descartan todos los datos de contacto y la dirección física.

STG_PRODUCTO: absorbe la tabla Categoria_producto. Conserva identificador, código, nombre, proveedor, cantidad en stock, precio de venta y precio proveedor. Incorpora el nombre de categoría desnormalizado y genera la columna calculada persistida STG_MARGEN_BRUTO como la diferencia entre precio de venta y precio proveedor.

STG_PEDIDO: absorbe los datos geográficos del cliente. Conserva identificador, fechas de pedido, esperada y entrega, estado e ID de cliente como FK. Incorpora nombre, ciudad y país del cliente desnormalizados. Genera tres columnas calculadas: STG_ANIO_PEDIDO, STG_MES_PEDIDO (ambas persistidas) y STG_DIAS_RETRASO (calculada, no persistida por depender de fecha nullable).

STG_DETALLE_PEDIDO: tabla de hechos central del modelo. Conserva FKs hacia pedido y producto, cantidad y precio unitario. Incorpora nombre de producto y categoría desnormalizados. Genera la columna calculada persistida STG_IMPORTE_LINEA como el producto de cantidad por precio unitario. Se descarta el número de línea.

STG_PAGO: se mantiene como tabla independiente por su diferente granularidad. Conserva FK de cliente, forma de pago, ID de transacción, fecha de pago y total. Incorpora el nombre del cliente desnormalizado. Genera STG_ANIO_PAGO y STG_MES_PAGO como columnas calculadas persistidas.

Construcción de las Consultas ETL

El proceso ETL se ejecuta en seis pasos consecutivos, correspondientes a cada tabla del Staging. La secuencia respeta el orden de creación de las tablas para mantener la integridad referencial. Cada paso utiliza sentencias INSERT INTO ... SELECT con JOINs hacia las tablas origen para resolver las desnormalizaciones en el momento de la carga.

Ejecución de la consulta para limpiar las tablas

Name	Value
Updated Rows	0
Execute time	0,026s
Start time	Thu Mar 26 20:36:05 COT 2026
Finish time	Thu Mar 26 20:36:05 COT 2026
Query	USE jardineria_staging;

```
-- — PASO 1: Deshabilitar todas las FK del Staging ————————————————
ALTER TABLE dbo.STG_PAGO          NOCHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_DETALLE_PEDIDO  NOCHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_PEDIDO        NOCHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_PRODUCTO      NOCHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_CLIENTE       NOCHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_EMPLEADO      NOCHECK CONSTRAINT ALL;
-- — PASO 2: Truncar en orden inverso de dependencias ————————————
TRUNCATE TABLE dbo.STG_PAGO;
TRUNCATE TABLE dbo.STG_DETALLE_PEDIDO;
TRUNCATE TABLE dbo.STG_PEDIDO;
TRUNCATE TABLE dbo.STG_PRODUCTO;
TRUNCATE TABLE dbo.STG_CLIENTE;
TRUNCATE TABLE dbo.STG_EMPLEADO;
-- — PASO 3: Rehabilitar todas las FK ————————————————————————
ALTER TABLE dbo.STG_PAGO          WITH CHECK CHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_DETALLE_PEDIDO  WITH CHECK CHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_PEDIDO        WITH CHECK CHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_PRODUCTO      WITH CHECK CHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_CLIENTE       WITH CHECK CHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_EMPLEADO      WITH CHECK CHECK CONSTRAINT ALL;
PRINT '>>> Staging limpiado. Listo para recarga.';
```

Figura 1

Limpieza de las tablas del staging antes de la carga de datos

Paso 1 — Carga de STG_EMPLEADO: realiza un LEFT JOIN entre las tablas empleado y oficina de la base jardinería. La función de concatenación con ISNULL garantiza que empleados sin apellido segundo queden correctamente registrados. Los campos de ciudad y país se normalizan con UPPER y LTRIM/RTRIM.

Name	Value
Updated Rows	31
Execute time	0,028s
Start time	Thu Mar 26 20:38:39 COT 2026
Finish time	Thu Mar 26 20:38:39 COT 2026
Query	-- Carga de las 5 tablas
	-- Paso 1 — STG_EMPLEADO (JOIN a oficina para desnormalizar)
	INSERT INTO dbo.STG_EMPLEADO (
	STG_ID_EMPLEADO, STG_NOMBRE_COMPLETO, STG_PUESTO,
	STG_OFI_CODIGO, STG_OFI_CIUDAD, STG_OFI_PAIS, STG_OFI_REGION,
	STG_FECHA_CARGA, STG_ORIGEN
	)
	SELECT
	e.ID_empleado,
	-- Consolidar nombre completo, eliminando apellido2 vacío:
	LTRIM(RTRIM(e.nombre + ' ' + e.apellido1
	+ ISNULL(' ' + NULLIF(LTRIM(RTRIM(e.apellido2)),''), ''))),
	LTRIM(RTRIM(e.puesto)),
	-- Desnormalizar oficina:
	LTRIM(RTRIM(o.Descripcion)),
	UPPER(LTRIM(RTRIM(o.ciudad))),
	UPPER(LTRIM(RTRIM(o.pais))),
	NULLIF(LTRIM(RTRIM(o.region)), ''),
	GETDATE(), 'jardineria'
	FROM jardineria.dbo.empleado e
	LEFT JOIN jardineria.dbo.oficina o ON e.ID_oficina = o.ID_oficina;
	PRINT ' Paso 1/5: STG_EMPLEADO — ' + CAST(@@ROWCOUNT AS VARCHAR) + ' filas.';

Figura 2

Carga de datos en la tabla STG_EMPLEADO

Paso 2 — Carga de STG_CLIENTE: ejecuta un LEFT JOIN encadenado entre cliente, empleado y oficina para resolver en un solo paso tanto los datos del cliente como los del representante asignado y su oficina. Los clientes sin representante asignado mantienen los campos STG_REP_* como NULL.

Name	Value
Updated Rows	36
Execute time	0,014s
Start time	Thu Mar 26 20:39:27 COT 2026
Finish time	Thu Mar 26 20:39:27 COT 2026
Query	-- Paso 2 — STG_CLIENTE (JOIN a empleado + oficina para desnormalizar representante)

```
INSERT INTO dbo.STG_CLIENTE (
   STG_ID_CLIENTE, STG_NOMBRE_CLIENTE, STG_CIUDAD, STG_REGION,
   STG_PAIS, STG_LIMITE_CREDITO,
   STG_REP_NOMBRE, STG_REP_PUESTO, STG_REP_OFI_CIUDAD, STG_REP_OFI_PAIS,
   STG_FECHA_CARGA, STG_ORIGEN
)
SELECT
   c.ID_cliente,
   LTRIM(RTRIM(c.nombre_cliente)),
   UPPER(LTRIM(RTRIM(c.ciudad))),
   NULLIF(LTRIM(RTRIM(c.region)), ''),
   UPPER(NULLIF(LTRIM(RTRIM(c.pais)), '')),
   ISNULL(c.limite_credito, 0),
   -- Desnormalizar representante de ventas:
   LTRIM(RTRIM(e.nombre + ' ' + e.apellido1
        + ISNULL(' ' + NULLIF(LTRIM(RTRIM(e.apellido2)),''), '')),
   LTRIM(RTRIM(e.puesto)),
   -- Desnormalizar oficina del representante:
   UPPER(LTRIM(RTRIM(o.ciudad))),
   UPPER(LTRIM(RTRIM(o.pais))),
   GETDATE(), 'jardineria'
FROM jardineria.dbo.cliente c
LEFT JOIN jardineria.dbo.empleado e ON c.ID_empleado_rep_ventas = e.ID_empleado
LEFT JOIN jardineria.dbo.oficina  o ON e.ID_oficina = o.ID_oficina;
PRINT '  Paso 2/5: STG_CLIENTE — ' + CAST(@@ROWCOUNT AS VARCHAR) + ' filas.';
```

Figura 3

Carga de datos en la tabla STG_CLIENTE

Paso 3 — Carga de STG_PRODUCTO: realiza un LEFT JOIN entre producto y Categoria_producto para incorporar el nombre de categoría. Los productos sin categoría asignada conservan STG_CATEGORIA como NULL.

Name	Value
Updated Rows	276
Execute time	0,021s
Start time	Thu Mar 26 20:40:03 COT 2026
Finish time	Thu Mar 26 20:40:03 COT 2026
Query	-- Paso 3 — STG_PRODUCTO (JOIN a Categoria_producto para desnormalizar)
	INSERT INTO dbo.STG_PRODUCTO (
	STG_ID_PRODUCTO, STG_CODIGO_PRODUCTO, STG_NOMBRE,
	STG_CATEGORIA, STG_PROVEEDOR, STG_CANTIDAD_STOCK,
	STG_PRECIO_VENTA, STG_PRECIO_PROVEEDOR,
	STG_FECHA_CARGA, STG_ORIGEN
	)
	SELECT
	p.ID_producto,
	UPPER(LTRIM(RTRIM(p.CodigoProducto))),
	LTRIM(RTRIM(p.nombre)),
	-- Desnormalizar nombre de categoría:
	LTRIM(RTRIM(cp.Desc_Categoria)),
	NULLIF(LTRIM(RTRIM(p.proveedor)), ''),
	ISNULL(p.cantidad_en_stock, 0),
	p.precio_venta,
	p.precio_proveedor,
	GETDATE(), 'jardineria'
	FROM jardineria.dbo.producto p
	LEFT JOIN jardineria.dbo.Categoria_producto cp ON p.Categoria = cp.Id_Categoria;
	PRINT ' Paso 3/5: STG_PRODUCTO — ' + CAST(@@ROWCOUNT AS VARCHAR) + ' filas.';

Figura 4

Carga de datos en la tabla STG_PRODUCTO

Paso 4 — Carga de STG_PEDIDO: ejecuta un LEFT JOIN entre pedido y cliente para desnormalizar los datos geográficos. El campo fecha_entrega se trata como DATE nullable para soportar pedidos aún no entregados.

Name	Value
Updated Rows	115
Execute time	0,011s
Start time	Thu Mar 26 20:40:41 COT 2026
Finish time	Thu Mar 26 20:40:41 COT 2026
Query	-- Paso 4 — STG_PEDIDO (JOIN a cliente para desnormalizar datos geográficos)
	INSERT INTO dbo.STG_PEDIDO (
	STG_ID_PEDIDO, STG_FECHA_PEDIDO, STG_FECHA_ESPERADA,
	STG_FECHA_ENTREGA, STG_ESTADO, STG_ID_CLIENTE,
	STG_CLI_NOMBRE, STG_CLI_CIUDAD, STG_CLI_PAIS,
	STG_FECHA_CARGA, STG_ORIGEN
	)
	SELECT
	p.ID_pedido,
	CAST(p.fecha_pedido AS DATE),
	CAST(p.fecha_esperada AS DATE),
	CAST(p.fecha_entrega AS DATE), -- puede ser NULL
	UPPER(LTRIM(RTRIM(p.estado))),
	p.ID_cliente,
	-- Desnormalizar datos del cliente:
	LTRIM(RTRIM(c.nombre_cliente)),
	UPPER(LTRIM(RTRIM(c.ciudad))),
	UPPER(NULLIF(LTRIM(RTRIM(c.pais)), '')),
	GETDATE(), 'jardineria'
	FROM jardineria.dbo.pedido p
	LEFT JOIN jardineria.dbo.cliente c ON p.ID_cliente = c.ID_cliente;
	PRINT ' Paso 4/5: STG_PEDIDO — ' + CAST(@@ROWCOUNT AS VARCHAR) + ' filas.';

Figura 5

Carga de datos en la tabla STG_PEDIDO

Paso 5 — Carga de STG_DETALLE_PEDIDO: realiza un LEFT JOIN doble entre detalle_pedido, producto y Categoria_producto. Incluye un filtro WHERE que descarta registros con cantidad o precio unitario igual a cero, garantizando la calidad de la tabla de hechos.

Name	Value
Updated Rows	318
Execute time	0,017s
Start time	Thu Mar 26 20:41:20 COT 2026
Finish time	Thu Mar 26 20:41:20 COT 2026
Query	-- Paso 5 — STG_DETALLE_PEDIDO (JOIN a producto + categoría para desnormalizar)

```
INSERT INTO dbo.STG_DETALLE_PEDIDO (
    STG_ID_PEDIDO, STG_ID_PRODUCTO, STG_CANTIDAD, STG_PRECIO_UNIDAD,
    STG_PRO_NOMBRE, STG_PRO_CATEGORIA,
    STG_FECHA_CARGA, STG_ORIGEN
)
SELECT
    dp.ID_pedido,
    dp.ID_producto,
    dp.cantidad,
    dp.precio_unidad,
    -- Desnormalizar nombre y categoría del producto:
    LTRIM(RTRIM(p.nombre)),
    LTRIM(RTRIM(cp.Desc_Categoria)),
    GETDATE(), 'jardineria'
FROM jardineria.dbo.detalle_pedido dp
LEFT JOIN jardineria.dbo.producto        p  ON dp.ID_producto = p.ID_producto
LEFT JOIN jardineria.dbo.Categoria_producto cp ON p.Categoria   = cp.Id_Categoria
WHERE dp.cantidad > 0 AND dp.precio_unidad > 0;
PRINT '  Paso 5/5: STG_DETALLE_PEDIDO — ' + CAST(@@ROWCOUNT AS VARCHAR) + ' filas.';
```

Figura 6

Carga de datos en la tabla STG_DETALLE_PEDIDO

Paso 6 — Carga de STG_PAGO: ejecuta un LEFT JOIN entre pago y cliente para incorporar el nombre del cliente. Incluye filtro WHERE que excluye pagos con total igual a cero.

Name	Value
Updated Rows	26
Execute time	0,011s
Start time	Thu Mar 26 20:41:51 COT 2026
Finish time	Thu Mar 26 20:41:51 COT 2026
Query	-- Paso 6 — STG_PAGO (JOIN a cliente para desnormalizar nombre)
	INSERT INTO dbo.STG_PAGO (
	STG_ID_CLIENTE, STG_CLI_NOMBRE, STG_FORMA_PAGO,
	STG_ID_TRANSACCION, STG_FECHA_PAGO, STG_TOTAL,
	STG_FECHA_CARGA, STG_ORIGEN
	)
	SELECT
	pg.ID_cliente,
	-- Desnormalizar nombre del cliente:
	LTRIM(RTRIM(c.nombre_cliente)),
	LTRIM(RTRIM(pg.forma_pago)),
	LTRIM(RTRIM(pg.id_transaccion)),
	CAST(pg.fecha_pago AS DATE),
	pg.total,
	GETDATE(), 'jardineria'
	FROM jardineria.dbo.pago pg
	LEFT JOIN jardineria.dbo.cliente c ON pg.ID_cliente = c.ID_cliente
	WHERE pg.total > 0;
	PRINT ' STG_PAGO — ' + CAST(@@ROWCOUNT AS VARCHAR) + ' filas.';
	PRINT '>>> Fase 3 completada: ETL ejecutado correctamente.';

Figura 7

Carga de datos en la tabla STG_PAGO

Validación de los Datos en el Staging

El proceso de validación comprende cuatro verificaciones sistemáticas que aseguran la calidad e integridad de los datos cargados:

Validación 1 — Conteo de registros: compara el número de filas entre cada tabla origen y su correspondiente tabla en el Staging. Para tablas fusionadas (STG_EMPLEADO, STG_CLIENTE, STG_PRODUCTO), el conteo de referencia es la tabla principal de la fusión,

ya que las tablas absorbidas solo aportan columnas, no filas adicionales. La diferencia esperada en todos los casos es cero.

```
-- 4.1 Conteo de Registros — Fuente vs. Staging
SELECT
    Tabla,
    MAX(CASE WHEN Origen = 'FUENTE'  THEN Total ELSE 0 END) AS Filas_Fuente,
    MAX(CASE WHEN Origen = 'STAGING' THEN Total ELSE 0 END) AS Filas_Staging,
    MAX(CASE WHEN Origen = 'FUENTE'  THEN Total ELSE 0 END)
  - MAX(CASE WHEN Origen = 'STAGING' THEN Total ELSE 0 END) AS Diferencia
FROM (
    SELECT 'empleado'           AS Tabla, 'FUENTE'  AS Origen, COUNT(*) AS
Total FROM jardineria.dbo.empleado            UNION ALL
    SELECT 'STG_EMPLEADO'       AS Tabla, 'STAGING' AS Origen, COUNT(*) AS
Total FROM jardineria_staging.dbo.STG_EMPLEADO UNION ALL
    SELECT 'cliente'            AS Tabla, 'FUENTE'  AS Origen, COUNT(*) AS
Total FROM jardineria.dbo.cliente              UNION ALL
    SELECT 'STG_CLIENTE'        AS Tabla, 'STAGING' AS Origen, COUNT(*) AS
Total FROM jardineria_staging.dbo.STG_CLIENTE  UNION ALL
    SELECT 'producto'           AS Tabla, 'FUENTE'  AS Origen, COUNT(*) AS
Total FROM jardineria.dbo.producto             UNION ALL
    SELECT 'STG_PRODUCTO'       AS Tabla, 'STAGING' AS Origen, COUNT(*) AS
Total FROM jardineria_staging.dbo.STG_PRODUCTO UNION ALL
    SELECT 'pedido'             AS Tabla, 'FUENTE'  AS Origen, COUNT(*) AS
Total FROM jardineria.dbo.pedido               UNION ALL
    SELECT 'STG_PEDIDO'         AS Tabla, 'STAGING' AS Origen, COUNT(*) AS
Total FROM jardineria_staging.dbo.STG_PEDIDO   UNION ALL
    SELECT 'detalle_pedido'     AS Tabla, 'FUENTE'  AS Origen, COUNT(*) AS
Total FROM jardineria.dbo.detalle_pedido               UNION ALL
    SELECT 'STG_DETALLE_PEDIDO' AS Tabla, 'STAGING' AS Origen, COUNT(*) AS
Total FROM jardineria_staging.dbo.STG_DETALLE_PEDIDO   UNION ALL
    SELECT 'pago'               AS Tabla, 'FUENTE'  AS Origen, COUNT(*) AS
Total FROM jardineria.dbo.pago                 UNION ALL
    SELECT 'STG_PAGO'           AS Tabla, 'STAGING' AS Origen, COUNT(*) AS
Total FROM jardineria_staging.dbo.STG_PAGO
) AS src
GROUP BY Tabla
ORDER BY Tabla;
GO
```

Validación 2 — Control de NULLs inesperados: verifica que los campos desnormalizados críticos no contengan valores nulos donde no deberían. Por ejemplo, STG_OFI_CIUDAD en STG_EMPLEADO, STG_CATEGORIA en STG_PRODUCTO y STG_CLI_NOMBRE en STG_PEDIDO. El resultado esperado es cero registros con NULL en estos campos.

Validación 3 — Totales financieros: compara la suma total de ingresos (cantidad × precio_unidad) entre la tabla detalle_pedido origen y STG_DETALLE_PEDIDO, y la suma de pagos entre la tabla pago origen y STG_PAGO. La diferencia esperada es cero en ambos casos.

Validación 4 — Coherencia de datos desnormalizados: verifica que los valores desnormalizados en el Staging sean consistentes entre sí. Por ejemplo, que el nombre de cliente en STG_PEDIDO coincida con el de STG_CLIENTE para el mismo ID de cliente, y que la categoría en STG_DETALLE_PEDIDO coincida con la de STG_PRODUCTO.

Scripts de Creación de la Base de Datos Staging

A continuación, se presenta el script DDL completo para la creación de la base de datos Staging y sus seis tablas. Este script debe ejecutarse antes de las sentencias ETL y en el orden indicado para garantizar la correcta resolución de las dependencias entre tablas.

Creación de la Base de Datos

```
IF NOT EXISTS (SELECT name FROM sys.databases WHERE name =
N'jardineria_staging')
    CREATE DATABASE jardineria_staging;
GO
USE jardineria_staging;
GO
```

Tabla STG_EMPLEADO

```
CREATE TABLE dbo.STG_EMPLEADO (
    STG_ID_EMPLEADO      INT            NOT NULL,
    STG_NOMBRE_COMPLETO  NVARCHAR(150)  NOT NULL,
    STG_PUESTO           NVARCHAR(50)   NULL,
    STG_OFI_CODIGO       NVARCHAR(10)   NULL,
    STG_OFI_CIUDAD       NVARCHAR(30)   NULL,
    STG_OFI_PAIS         NVARCHAR(50)   NULL,
    STG_OFI_REGION       NVARCHAR(50)   NULL,
    STG_FECHA_CARGA      DATETIME2      NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN           NVARCHAR(50)   NOT NULL DEFAULT 'jardineria',
    CONSTRAINT PK_STG_EMPLEADO PRIMARY KEY (STG_ID_EMPLEADO)
);
```

Tabla STG_CLIENTE

```
CREATE TABLE dbo.STG_CLIENTE (
    STG_ID_CLIENTE       INT            NOT NULL,
    STG_NOMBRE_CLIENTE   NVARCHAR(50)   NOT NULL,
    STG_CIUDAD           NVARCHAR(50)   NULL,
    STG_REGION           NVARCHAR(50)   NULL,
    STG_PAIS             NVARCHAR(50)   NULL,
    STG_LIMITE_CREDITO   NUMERIC(15,2)  NULL,
    STG_REP_NOMBRE       NVARCHAR(150)  NULL,
    STG_REP_PUESTO       NVARCHAR(50)   NULL,
    STG_REP_OFI_CIUDAD   NVARCHAR(30)   NULL,
    STG_REP_OFI_PAIS     NVARCHAR(50)   NULL,
    STG_FECHA_CARGA      DATETIME2      NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN           NVARCHAR(50)   NOT NULL DEFAULT 'jardineria',
```

```
    CONSTRAINT PK_STG_CLIENTE PRIMARY KEY (STG_ID_CLIENTE)
);
```

Tabla STG_PRODUCTO

```
CREATE TABLE dbo.STG_PRODUCTO (
    STG_ID_PRODUCTO      INT            NOT NULL,
    STG_CODIGO_PRODUCTO  NVARCHAR(15)   NOT NULL,
    STG_NOMBRE           NVARCHAR(70)   NOT NULL,
    STG_CATEGORIA        NVARCHAR(50)   NULL,
    STG_PROVEEDOR        NVARCHAR(50)   NULL,
    STG_CANTIDAD_STOCK   INT            NOT NULL DEFAULT 0,
    STG_PRECIO_VENTA     NUMERIC(15,2)  NOT NULL,
    STG_PRECIO_PROVEEDOR NUMERIC(15,2)  NULL,
    STG_MARGEN_BRUTO     AS (STG_PRECIO_VENTA - ISNULL(STG_PRECIO_PROVEEDOR,0))
PERSISTED,
    STG_FECHA_CARGA      DATETIME2      NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN           NVARCHAR(50)   NOT NULL DEFAULT 'jardineria',
    CONSTRAINT PK_STG_PRODUCTO PRIMARY KEY (STG_ID_PRODUCTO),
    CONSTRAINT UQ_STG_COD_PROD UNIQUE (STG_CODIGO_PRODUCTO)
);
```

Tabla STG_PEDIDO

```
CREATE TABLE dbo.STG_PEDIDO (
    STG_ID_PEDIDO        INT            NOT NULL,
    STG_FECHA_PEDIDO     DATE           NOT NULL,
    STG_FECHA_ESPERADA   DATE           NOT NULL,
    STG_FECHA_ENTREGA    DATE           NULL,
    STG_ESTADO           NVARCHAR(20)   NOT NULL,
    STG_ID_CLIENTE       INT            NOT NULL,
    STG_CLI_NOMBRE       NVARCHAR(50)   NULL,
    STG_CLI_CIUDAD       NVARCHAR(50)   NULL,
    STG_CLI_PAIS         NVARCHAR(50)   NULL,
    STG_ANIO_PEDIDO      AS (YEAR(STG_FECHA_PEDIDO)) PERSISTED,
```

```
    STG_MES_PEDIDO       AS (MONTH(STG_FECHA_PEDIDO)) PERSISTED,
    STG_DIAS_RETRASO     AS (CASE WHEN STG_FECHA_ENTREGA IS NOT NULL
                            THEN
DATEDIFF(DAY,STG_FECHA_ESPERADA,STG_FECHA_ENTREGA)
                            ELSE NULL END),
    STG_FECHA_CARGA      DATETIME2      NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN           NVARCHAR(50)   NOT NULL DEFAULT 'jardineria',
    CONSTRAINT PK_STG_PEDIDO PRIMARY KEY (STG_ID_PEDIDO),
    CONSTRAINT FK_STG_PED_CLI FOREIGN KEY (STG_ID_CLIENTE)
        REFERENCES dbo.STG_CLIENTE (STG_ID_CLIENTE)
);
```

Tabla STG_DETALLE_PEDIDO

```
CREATE TABLE dbo.STG_DETALLE_PEDIDO (
    STG_ID_DETALLE       INT IDENTITY(1,1) NOT NULL,
    STG_ID_PEDIDO        INT            NOT NULL,
    STG_ID_PRODUCTO      INT            NOT NULL,
    STG_CANTIDAD         INT            NOT NULL,
    STG_PRECIO_UNIDAD    NUMERIC(15,2)  NOT NULL,
    STG_PRO_NOMBRE       NVARCHAR(70)   NULL,
    STG_PRO_CATEGORIA    NVARCHAR(50)   NULL,
    STG_IMPORTE_LINEA    AS (STG_CANTIDAD * STG_PRECIO_UNIDAD) PERSISTED,
    STG_FECHA_CARGA      DATETIME2      NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN           NVARCHAR(50)   NOT NULL DEFAULT 'jardineria',
    CONSTRAINT PK_STG_DETALLE PRIMARY KEY (STG_ID_DETALLE),
    CONSTRAINT FK_STG_DET_PED FOREIGN KEY (STG_ID_PEDIDO)
        REFERENCES dbo.STG_PEDIDO (STG_ID_PEDIDO),
    CONSTRAINT FK_STG_DET_PRO FOREIGN KEY (STG_ID_PRODUCTO)
        REFERENCES dbo.STG_PRODUCTO (STG_ID_PRODUCTO)
);
```

Tabla STG_PAGO

```
CREATE TABLE dbo.STG_PAGO (
    STG_ID_PAGO          INT IDENTITY(1,1) NOT NULL,
    STG_ID_CLIENTE       INT            NOT NULL,
```

```
    STG_CLI_NOMBRE       NVARCHAR(50)   NULL,
    STG_FORMA_PAGO       NVARCHAR(40)   NOT NULL,
    STG_ID_TRANSACCION   NVARCHAR(50)   NOT NULL,
    STG_FECHA_PAGO       DATE           NOT NULL,
    STG_TOTAL            NUMERIC(15,2)  NOT NULL,
    STG_ANIO_PAGO        AS (YEAR(STG_FECHA_PAGO)) PERSISTED,
    STG_MES_PAGO         AS (MONTH(STG_FECHA_PAGO)) PERSISTED,
    STG_FECHA_CARGA      DATETIME2      NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN           NVARCHAR(50)   NOT NULL DEFAULT 'jardineria',
    CONSTRAINT PK_STG_PAGO PRIMARY KEY (STG_ID_PAGO),
    CONSTRAINT UQ_STG_TRANSACCION UNIQUE (STG_ID_TRANSACCION),
    CONSTRAINT FK_STG_PAG_CLI FOREIGN KEY (STG_ID_CLIENTE)
        REFERENCES dbo.STG_CLIENTE (STG_ID_CLIENTE)
);
```

Resultados de las consultas SQL para la creación de la BD Jardinería Staging

Estadísticas 1

Name	Value
Updated Rows	0
Execute time	0,305s
Start time	Thu Mar 26 20:07:47 COT 2026
Finish time	Thu Mar 26 20:07:47 COT 2026
Query	CREATE DATABASE jardineria_staging;

Figura 8

Creación de la base de datos jardineria_staging

Estadísticas 1

Name	Value
Updated Rows	-1
Execute time	0,005s
Start time	Thu Mar 26 20:09:12 COT 2026
Finish time	Thu Mar 26 20:09:12 COT 2026
Query	USE jardineria_staging;

Figura 9

Ejecución del comando USE en la base de datos jardineria_staging

Estadísticas 1

Name	Value
Updated Rows	0
Execute time	0,01s
Start time	Thu Mar 26 20:09:58 COT 2026
Finish time	Thu Mar 26 20:09:58 COT 2026
Query	--Tabla 1 de 5 — STG_EMPLEADO (absorbe oficina)

```
IF OBJECT_ID('dbo.STG_EMPLEADO','U') IS NOT NULL DROP TABLE dbo.STG_EMPLEADO;
CREATE TABLE dbo.STG_EMPLEADO (
    STG_ID_EMPLEADO     INT            NOT NULL,
    STG_NOMBRE_COMPLETO NVARCHAR(150)   NOT NULL,   -- nombre + apellido1 + apellido2
    STG_PUESTO          NVARCHAR(50)    NULL,
    -- Campos desnormalizados de oficina:
    STG_OFI_CODIGO      NVARCHAR(10)    NULL,       -- ej. MAD-ES, BCN-ES
    STG_OFI_CIUDAD      NVARCHAR(30)    NULL,
    STG_OFI_PAIS        NVARCHAR(50)    NULL,
    STG_OFI_REGION      NVARCHAR(50)    NULL,       -- ej. EMEA, APAC
    -- Auditoría ETL:
    STG_FECHA_CARGA     DATETIME2       NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN          NVARCHAR(50)    NOT NULL DEFAULT 'jardineria',
    CONSTRAINT PK_STG_EMPLEADO PRIMARY KEY (STG_ID_EMPLEADO)
);
```

Figura 10

Creación de la tabla STG_EMPLEADO

Estadísticas 1

Name	Value
Updated Rows	0
Execute time	0,009s
Start time	Thu Mar 26 20:11:07 COT 2026
Finish time	Thu Mar 26 20:11:07 COT 2026
Query	-- Tabla 2 de 5 — STG_CLIENTE (absorbe representante de ventas)

```
IF OBJECT_ID('dbo.STG_CLIENTE','U') IS NOT NULL DROP TABLE dbo.STG_CLIENTE;
CREATE TABLE dbo.STG_CLIENTE (
  STG_ID_CLIENTE      INT            NOT NULL,
  STG_NOMBRE_CLIENTE  NVARCHAR(50)   NOT NULL,
  STG_CIUDAD          NVARCHAR(50)   NULL,
  STG_REGION          NVARCHAR(50)   NULL,
  STG_PAIS            NVARCHAR(50)   NULL,
  STG_LIMITE_CREDITO  NUMERIC(15,2)  NULL,
  -- Campos desnormalizados del representante de ventas:
  STG_REP_NOMBRE      NVARCHAR(150)  NULL,     -- nombre completo del representante
  STG_REP_PUESTO      NVARCHAR(50)   NULL,
  STG_REP_OFI_CIUDAD  NVARCHAR(30)   NULL,     -- ciudad de la oficina del representante
  STG_REP_OFI_PAIS    NVARCHAR(50)   NULL,
  -- Auditoría ETL:
  STG_FECHA_CARGA     DATETIME2      NOT NULL DEFAULT GETDATE(),
  STG_ORIGEN          NVARCHAR(50)   NOT NULL DEFAULT 'jardineria',
  CONSTRAINT PK_STG_CLIENTE PRIMARY KEY (STG_ID_CLIENTE)
);
```

Figura 11

Creación de la tabla STG_CLIENTE

Name	Value
Updated Rows	0
Execute time	0,015s
Start time	Thu Mar 26 20:12:28 COT 2026
Finish time	Thu Mar 26 20:12:28 COT 2026
Query	-- Tabla 3 de 5 — STG_PRODUCTO (absorbe categoría)

```
IF OBJECT_ID('dbo.STG_PRODUCTO','U') IS NOT NULL DROP TABLE dbo.STG_PRODUCTO;
CREATE TABLE dbo.STG_PRODUCTO (
  STG_ID_PRODUCTO      INT            NOT NULL,
  STG_CODIGO_PRODUCTO  NVARCHAR(15)   NOT NULL,
  STG_NOMBRE           NVARCHAR(70)   NOT NULL,
  -- Campo desnormalizado de categoría:
  STG_CATEGORIA        NVARCHAR(50)   NULL,     -- Desc_Categoria de Categoria_producto
  STG_PROVEEDOR        NVARCHAR(50)   NULL,
  STG_CANTIDAD_STOCK   INT            NOT NULL DEFAULT 0,
  STG_PRECIO_VENTA     NUMERIC(15,2)  NOT NULL,
  STG_PRECIO_PROVEEDOR NUMERIC(15,2)  NULL,
  -- Columna calculada persistida:
  STG_MARGEN_BRUTO     AS (STG_PRECIO_VENTA
                   - ISNULL(STG_PRECIO_PROVEEDOR, 0)) PERSISTED,
  -- Auditoría ETL:
  STG_FECHA_CARGA      DATETIME2      NOT NULL DEFAULT GETDATE(),
  STG_ORIGEN           NVARCHAR(50)   NOT NULL DEFAULT 'jardineria',
  CONSTRAINT PK_STG_PRODUCTO  PRIMARY KEY (STG_ID_PRODUCTO),
  CONSTRAINT UQ_STG_COD_PROD  UNIQUE (STG_CODIGO_PRODUCTO)
);
```

Figura 12

Creación de la tabla STG_PRODUCTO

Name	Value
Updated Rows	0
Execute time	0,009s
Start time	Thu Mar 26 20:13:30 COT 2026
Finish time	Thu Mar 26 20:13:30 COT 2026
Query	-- Tabla 4 de 5 — STG_PEDIDO (absorbe datos geográficos del cliente)

```
IF OBJECT_ID('dbo.STG_PEDIDO','U') IS NOT NULL DROP TABLE dbo.STG_PEDIDO;
CREATE TABLE dbo.STG_PEDIDO (
    STG_ID_PEDIDO       INT             NOT NULL,
    STG_FECHA_PEDIDO    DATE            NOT NULL,
    STG_FECHA_ESPERADA  DATE            NOT NULL,
    STG_FECHA_ENTREGA   DATE            NULL,
    STG_ESTADO          NVARCHAR(20)    NOT NULL,
    STG_ID_CLIENTE      INT             NOT NULL,
    -- Campos desnormalizados del cliente:
    STG_CLI_NOMBRE      NVARCHAR(50)    NULL,     -- nombre del cliente
    STG_CLI_CIUDAD      NVARCHAR(50)    NULL,     -- ciudad del cliente
    STG_CLI_PAIS        NVARCHAR(50)    NULL,     -- país del cliente
    -- Columnas calculadas persistidas:
    STG_ANIO_PEDIDO     AS (YEAR(STG_FECHA_PEDIDO)) PERSISTED,
    STG_MES_PEDIDO      AS (MONTH(STG_FECHA_PEDIDO)) PERSISTED,
    -- Columna calculada no persistida (depende de fecha_entrega nullable):
    STG_DIAS_RETRASO    AS (CASE WHEN STG_FECHA_ENTREGA IS NOT NULL
                         THEN DATEDIFF(DAY, STG_FECHA_ESPERADA, STG_FECHA_ENTREGA)
                         ELSE NULL END),
    -- Auditoría ETL:
    STG_FECHA_CARGA     DATETIME2       NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN          NVARCHAR(50)    NOT NULL DEFAULT 'jardineria',
    CONSTRAINT PK_STG_PEDIDO PRIMARY KEY (STG_ID_PEDIDO),
    CONSTRAINT FK_STG_PED_CLI FOREIGN KEY (STG_ID_CLIENTE)
      REFERENCES dbo.STG_CLIENTE (STG_ID_CLIENTE)
);
```

Figura 13

Creación de la tabla STG_PEDIDO

Name	Value
Updated Rows	0
Execute time	0,012s
Start time	Thu Mar 26 20:14:09 COT 2026
Finish time	Thu Mar 26 20:14:09 COT 2026
Query	-- Tabla 5 de 5 — STG_DETALLE_PEDIDO (tabla de hechos, absorbe nombre y categoría de producto)

```
IF OBJECT_ID('dbo.STG_DETALLE_PEDIDO','U') IS NOT NULL DROP TABLE dbo.STG_DETALLE_PEDIDO;
CREATE TABLE dbo.STG_DETALLE_PEDIDO (
    STG_ID_DETALLE      INT             IDENTITY(1,1) NOT NULL,
    STG_ID_PEDIDO       INT             NOT NULL,
    STG_ID_PRODUCTO     INT             NOT NULL,
    STG_CANTIDAD        INT             NOT NULL,
    STG_PRECIO_UNIDAD   NUMERIC(15,2)   NOT NULL,
    -- Campos desnormalizados del producto:
    STG_PRO_NOMBRE      NVARCHAR(70)    NULL,     -- nombre del producto
    STG_PRO_CATEGORIA   NVARCHAR(50)    NULL,     -- categoría del producto
    -- Columna calculada persistida:
    STG_IMPORTE_LINEA   AS (STG_CANTIDAD * STG_PRECIO_UNIDAD) PERSISTED,
    -- Auditoría ETL:
    STG_FECHA_CARGA     DATETIME2       NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN          NVARCHAR(50)    NOT NULL DEFAULT 'jardineria',
    CONSTRAINT PK_STG_DETALLE PRIMARY KEY (STG_ID_DETALLE),
    CONSTRAINT FK_STG_DET_PED FOREIGN KEY (STG_ID_PEDIDO)
      REFERENCES dbo.STG_PEDIDO (STG_ID_PEDIDO),
    CONSTRAINT FK_STG_DET_PRO FOREIGN KEY (STG_ID_PRODUCTO)
      REFERENCES dbo.STG_PRODUCTO (STG_ID_PRODUCTO)
);
```

Figura 14

Creación de la tabla STG_DETALLE_PEDIDO

Name	Value
Updated Rows	0
Execute time	0,01s
Start time	Thu Mar 26 20:14:33 COT 2026
Finish time	Thu Mar 26 20:14:33 COT 2026
Query	-- Tabla STG_PAGO (absorbe nombre del cliente)
	IF OBJECT_ID('dbo.STG_PAGO','U') IS NOT NULL DROP TABLE dbo.STG_PAGO;
	CREATE TABLE dbo.STG_PAGO (
	STG_ID_PAGO INT IDENTITY(1,1) NOT NULL,
	STG_ID_CLIENTE INT NOT NULL,
	STG_CLI_NOMBRE NVARCHAR(50) NULL, -- nombre desnormalizado del cliente
	STG_FORMA_PAGO NVARCHAR(40) NOT NULL,
	STG_ID_TRANSACCION NVARCHAR(50) NOT NULL,
	STG_FECHA_PAGO DATE NOT NULL,
	STG_TOTAL NUMERIC(15,2) NOT NULL,
	-- Columnas calculadas persistidas:
	STG_ANIO_PAGO AS (YEAR(STG_FECHA_PAGO)) PERSISTED,
	STG_MES_PAGO AS (MONTH(STG_FECHA_PAGO)) PERSISTED,
	-- Auditoría ETL:
	STG_FECHA_CARGA DATETIME2 NOT NULL DEFAULT GETDATE(),
	STG_ORIGEN NVARCHAR(50) NOT NULL DEFAULT 'jardineria',
	CONSTRAINT PK_STG_PAGO PRIMARY KEY (STG_ID_PAGO),
	CONSTRAINT UQ_STG_TRANSACCION UNIQUE (STG_ID_TRANSACCION),
	CONSTRAINT FK_STG_PAG_CLI FOREIGN KEY (STG_ID_CLIENTE)
	REFERENCES dbo.STG_CLIENTE (STG_ID_CLIENTE)
	);

Figura 15

Creación de la tabla STG_PAGO

Conclusiones

La construcción de la base de datos Staging a partir de la base de datos Jardinería demostró que la aplicación de técnicas de desnormalización controlada y limpieza de campos operacionales genera beneficios cuantificables en la eficiencia de las consultas analíticas. La reducción de ocho a cinco entidades en el modelo, junto con la incorporación de siete columnas calculadas persistidas, permite responder preguntas de negocio complejas con un número significativamente menor de JOINs: de cinco JOINs que requeriría el modelo transaccional original para el análisis de ventas por oficina, a dos JOINs en el modelo Staging implementado.

El proceso de auditoría campo por campo reveló que 25 de los 65 campos originales (un 38.5%) corresponden a datos operacionales sin valor analítico, lo que justifica plenamente la existencia de una capa Staging diferenciada del sistema transaccional. La eliminación de estos campos reduce el volumen de datos procesados y simplifica el mantenimiento del modelo analítico.

Los procedimientos de validación implementados en cuatro etapas —conteo de registros, control de NULLs, verificación de totales financieros y coherencia de datos desnormalizados— proporcionan una garantía sistemática de la calidad de los datos cargados. La implementación de campos de auditoría ETL en todas las tablas (STG_FECHA_CARGA y STG_ORIGEN) facilita el monitoreo y la trazabilidad del proceso de carga.

Finalmente, la estrategia de backup implementada para ambas bases de datos —jardinería antes del ETL y jardineria_staging después de la validación— asegura la recuperabilidad del sistema ante eventuales fallas, complementando el ciclo completo de gestión de datos desde la fuente hasta la capa analítica.

Anexos

Anexo A. Script Completo de Creación de la Base de Datos Staging

El script completo para la creación de la base de datos Staging (DDL) se presenta a continuación. Este incluye la creación de la base de datos *jardineria_staging* y la definición de las seis tablas que conforman el modelo, junto con sus respectivas claves primarias, foráneas, columnas calculadas y campos de auditoría ETL, se presenta en el Archivo 1 adjunto: https://drive.google.com/file/d/1ubWCBid4t0F88nnwQT_R29cM8xId42Tm/view?usp=drive_link

```
--Crear la Base de Datos Staging
IF NOT EXISTS (SELECT name FROM sys.databases WHERE name =
N'jardineria_staging')
    CREATE DATABASE jardineria_staging;

USE jardineria_staging;

--Tabla 1 de 5 — STG_EMPLEADO  (absorbe oficina)
IF OBJECT_ID('dbo.STG_EMPLEADO','U') IS NOT NULL DROP TABLE dbo.STG_EMPLEADO;

CREATE TABLE dbo.STG_EMPLEADO (
    STG_ID_EMPLEADO     INT              NOT NULL,
    STG_NOMBRE_COMPLETO NVARCHAR(150)    NOT NULL,   -- nombre + apellido1 +
apellido2
    STG_PUESTO          NVARCHAR(50)     NULL,
    -- Campos desnormalizados de oficina:
    STG_OFI_CODIGO      NVARCHAR(10)     NULL,       -- ej. MAD-ES, BCN-ES
    STG_OFI_CIUDAD      NVARCHAR(30)     NULL,
    STG_OFI_PAIS        NVARCHAR(50)     NULL,
    STG_OFI_REGION      NVARCHAR(50)     NULL,       -- ej. EMEA, APAC
    -- Auditoría ETL:
    STG_FECHA_CARGA     DATETIME2        NOT NULL DEFAULT GETDATE(),
```

```
    STG_ORIGEN          NVARCHAR(50)     NOT NULL DEFAULT 'jardineria',
    CONSTRAINT PK_STG_EMPLEADO PRIMARY KEY (STG_ID_EMPLEADO)
);

-- Tabla 2 de 5 — STG_CLIENTE  (absorbe representante de ventas)
IF OBJECT_ID('dbo.STG_CLIENTE','U') IS NOT NULL DROP TABLE dbo.STG_CLIENTE;

CREATE TABLE dbo.STG_CLIENTE (
    STG_ID_CLIENTE      INT              NOT NULL,
    STG_NOMBRE_CLIENTE  NVARCHAR(50)     NOT NULL,
    STG_CIUDAD          NVARCHAR(50)     NULL,
    STG_REGION          NVARCHAR(50)     NULL,
    STG_PAIS            NVARCHAR(50)     NULL,
    STG_LIMITE_CREDITO  NUMERIC(15,2)    NULL,
    -- Campos desnormalizados del representante de ventas:
    STG_REP_NOMBRE      NVARCHAR(150)    NULL,       -- nombre completo del
representante
    STG_REP_PUESTO      NVARCHAR(50)     NULL,
    STG_REP_OFI_CIUDAD  NVARCHAR(30)     NULL,       -- ciudad de la oficina
del representante
    STG_REP_OFI_PAIS    NVARCHAR(50)     NULL,
    -- Auditoría ETL:
    STG_FECHA_CARGA     DATETIME2        NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN          NVARCHAR(50)     NOT NULL DEFAULT 'jardineria',
    CONSTRAINT PK_STG_CLIENTE PRIMARY KEY (STG_ID_CLIENTE)
);

-- Tabla 3 de 5 — STG_PRODUCTO  (absorbe categoría)
IF OBJECT_ID('dbo.STG_PRODUCTO','U') IS NOT NULL DROP TABLE dbo.STG_PRODUCTO;

CREATE TABLE dbo.STG_PRODUCTO (
    STG_ID_PRODUCTO      INT              NOT NULL,
    STG_CODIGO_PRODUCTO  NVARCHAR(15)     NOT NULL,
    STG_NOMBRE           NVARCHAR(70)     NOT NULL,
    -- Campo desnormalizado de categoría:
    STG_CATEGORIA        NVARCHAR(50)     NULL,       -- Desc_Categoria de
Categoria_producto
    STG_PROVEEDOR        NVARCHAR(50)     NULL,
    STG_CANTIDAD_STOCK   INT              NOT NULL DEFAULT 0,
    STG_PRECIO_VENTA     NUMERIC(15,2)    NOT NULL,
    STG_PRECIO_PROVEEDOR NUMERIC(15,2)    NULL,
    -- Columna calculada persistida:
```

```
    STG_MARGEN_BRUTO     AS (STG_PRECIO_VENTA
                         - ISNULL(STG_PRECIO_PROVEEDOR, 0)) PERSISTED,
    -- Auditoría ETL:
    STG_FECHA_CARGA      DATETIME2        NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN           NVARCHAR(50)     NOT NULL DEFAULT 'jardineria',
    CONSTRAINT PK_STG_PRODUCTO  PRIMARY KEY (STG_ID_PRODUCTO),
    CONSTRAINT UQ_STG_COD_PROD  UNIQUE (STG_CODIGO_PRODUCTO)
);

-- Tabla 4 de 5 — STG_PEDIDO  (absorbe datos geográficos del cliente)
IF OBJECT_ID('dbo.STG_PEDIDO','U') IS NOT NULL DROP TABLE dbo.STG_PEDIDO;

CREATE TABLE dbo.STG_PEDIDO (
    STG_ID_PEDIDO       INT              NOT NULL,
    STG_FECHA_PEDIDO    DATE             NOT NULL,
    STG_FECHA_ESPERADA  DATE             NOT NULL,
    STG_FECHA_ENTREGA   DATE             NULL,
    STG_ESTADO          NVARCHAR(20)     NOT NULL,
    STG_ID_CLIENTE      INT              NOT NULL,
    -- Campos desnormalizados del cliente:
    STG_CLI_NOMBRE      NVARCHAR(50)     NULL,       -- nombre del cliente
    STG_CLI_CIUDAD      NVARCHAR(50)     NULL,       -- ciudad del cliente
    STG_CLI_PAIS        NVARCHAR(50)     NULL,       -- país del cliente
    -- Columnas calculadas persistidas:
    STG_ANIO_PEDIDO     AS (YEAR(STG_FECHA_PEDIDO))  PERSISTED,
    STG_MES_PEDIDO      AS (MONTH(STG_FECHA_PEDIDO)) PERSISTED,
    -- Columna calculada no persistida (depende de fecha_entrega nullable):
    STG_DIAS_RETRASO    AS (CASE WHEN STG_FECHA_ENTREGA IS NOT NULL
                                THEN DATEDIFF(DAY, STG_FECHA_ESPERADA,
STG_FECHA_ENTREGA)
                                ELSE NULL END),
    -- Auditoría ETL:
    STG_FECHA_CARGA     DATETIME2        NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN          NVARCHAR(50)     NOT NULL DEFAULT 'jardineria',
    CONSTRAINT PK_STG_PEDIDO  PRIMARY KEY (STG_ID_PEDIDO),
    CONSTRAINT FK_STG_PED_CLI FOREIGN KEY (STG_ID_CLIENTE)
        REFERENCES dbo.STG_CLIENTE (STG_ID_CLIENTE)
);

-- Tabla 5 de 5 — STG_DETALLE_PEDIDO  (tabla de hechos, absorbe nombre y
categoría de producto)
```

```
IF OBJECT_ID('dbo.STG_DETALLE_PEDIDO','U') IS NOT NULL DROP TABLE
dbo.STG_DETALLE_PEDIDO;

CREATE TABLE dbo.STG_DETALLE_PEDIDO (
    STG_ID_DETALLE      INT              IDENTITY(1,1) NOT NULL,
    STG_ID_PEDIDO       INT              NOT NULL,
    STG_ID_PRODUCTO     INT              NOT NULL,
    STG_CANTIDAD        INT              NOT NULL,
    STG_PRECIO_UNIDAD   NUMERIC(15,2)    NOT NULL,
    -- Campos desnormalizados del producto:
    STG_PRO_NOMBRE      NVARCHAR(70)     NULL,       -- nombre del producto
    STG_PRO_CATEGORIA   NVARCHAR(50)     NULL,       -- categoría del producto
    -- Columna calculada persistida:
    STG_IMPORTE_LINEA   AS (STG_CANTIDAD * STG_PRECIO_UNIDAD) PERSISTED,
    -- Auditoría ETL:
    STG_FECHA_CARGA     DATETIME2        NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN          NVARCHAR(50)     NOT NULL DEFAULT 'jardineria',
    CONSTRAINT PK_STG_DETALLE   PRIMARY KEY (STG_ID_DETALLE),
    CONSTRAINT FK_STG_DET_PED   FOREIGN KEY (STG_ID_PEDIDO)
        REFERENCES dbo.STG_PEDIDO (STG_ID_PEDIDO),
    CONSTRAINT FK_STG_DET_PRO   FOREIGN KEY (STG_ID_PRODUCTO)
        REFERENCES dbo.STG_PRODUCTO (STG_ID_PRODUCTO)
);

-- Tabla STG_PAGO  (absorbe nombre del cliente)
IF OBJECT_ID('dbo.STG_PAGO','U') IS NOT NULL DROP TABLE dbo.STG_PAGO;

CREATE TABLE dbo.STG_PAGO (
    STG_ID_PAGO         INT              IDENTITY(1,1) NOT NULL,
    STG_ID_CLIENTE      INT              NOT NULL,
    STG_CLI_NOMBRE      NVARCHAR(50)     NULL,       -- nombre desnormalizado
del cliente
    STG_FORMA_PAGO      NVARCHAR(40)     NOT NULL,
    STG_ID_TRANSACCION  NVARCHAR(50)     NOT NULL,
    STG_FECHA_PAGO      DATE             NOT NULL,
    STG_TOTAL           NUMERIC(15,2)    NOT NULL,
    -- Columnas calculadas persistidas:
    STG_ANIO_PAGO       AS (YEAR(STG_FECHA_PAGO))  PERSISTED,
    STG_MES_PAGO        AS (MONTH(STG_FECHA_PAGO)) PERSISTED,
    -- Auditoría ETL:
    STG_FECHA_CARGA     DATETIME2        NOT NULL DEFAULT GETDATE(),
    STG_ORIGEN          NVARCHAR(50)     NOT NULL DEFAULT 'jardineria',
```

```
    CONSTRAINT PK_STG_PAGO        PRIMARY KEY (STG_ID_PAGO),
    CONSTRAINT UQ_STG_TRANSACCION UNIQUE (STG_ID_TRANSACCION),
    CONSTRAINT FK_STG_PAG_CLI     FOREIGN KEY (STG_ID_CLIENTE)
        REFERENCES dbo.STG_CLIENTE (STG_ID_CLIENTE)
);
```

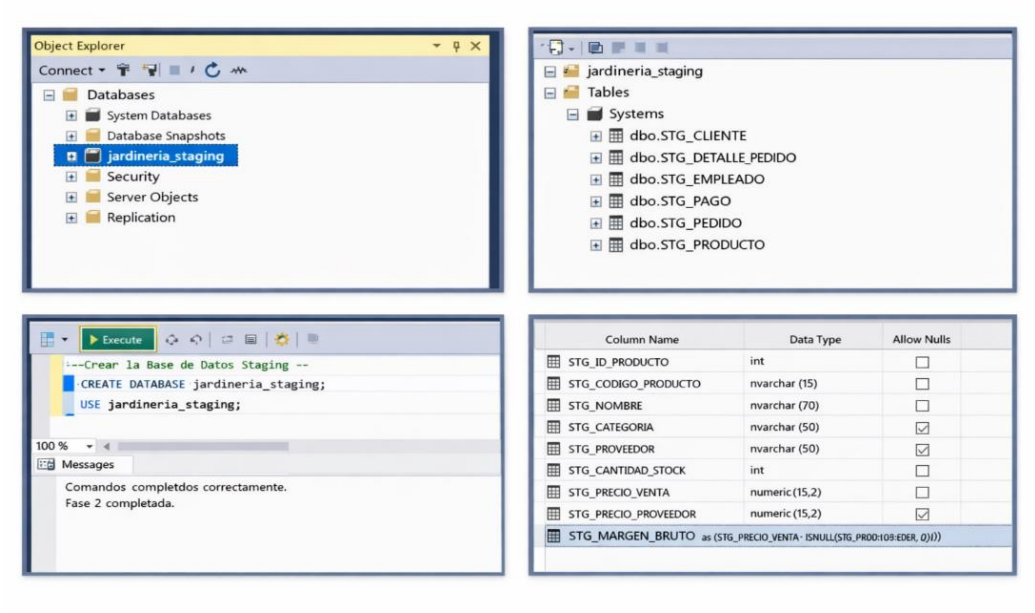

Figura 16

Ejecución completa del script de creación del staging

Anexo B. Script ETL — Sentencias de Carga

Las sentencias de extracción, transformación y carga (ETL) que trasladan los registros desde la base de datos Jardinería hacia el Staging, incluyendo los seis pasos de carga y las cuatro validaciones, se presentan en el Archivo 2 adjunto:

https://drive.google.com/file/d/1qQh4SV9vhknEEct9zJPZ0ozXFYEdzehD/view?usp=drive_link

```
--Fase 3 — ETL: Extracción, Transformación y Carga

-- Limpiar el Staging (TRUNCATE en orden inverso)
```

```
USE jardineria_staging;

-- ── PASO 1: Deshabilitar todas las FK del Staging ──────────────────────
ALTER TABLE dbo.STG_PAGO            NOCHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_DETALLE_PEDIDO  NOCHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_PEDIDO          NOCHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_PRODUCTO        NOCHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_CLIENTE         NOCHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_EMPLEADO        NOCHECK CONSTRAINT ALL;

-- ── PASO 2: Truncar en orden inverso de dependencias ──────────────────────
TRUNCATE TABLE dbo.STG_PAGO;
TRUNCATE TABLE dbo.STG_DETALLE_PEDIDO;
TRUNCATE TABLE dbo.STG_PEDIDO;
TRUNCATE TABLE dbo.STG_PRODUCTO;
TRUNCATE TABLE dbo.STG_CLIENTE;
TRUNCATE TABLE dbo.STG_EMPLEADO;

-- ── PASO 3: Rehabilitar todas las FK ──────────────────────────────────
ALTER TABLE dbo.STG_PAGO            WITH CHECK CHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_DETALLE_PEDIDO  WITH CHECK CHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_PEDIDO          WITH CHECK CHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_PRODUCTO        WITH CHECK CHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_CLIENTE         WITH CHECK CHECK CONSTRAINT ALL;
ALTER TABLE dbo.STG_EMPLEADO        WITH CHECK CHECK CONSTRAINT ALL;

PRINT '>>> Staging limpiado. Listo para recarga.';

-- Carga de las 5 tablas
-- Paso 1 — STG_EMPLEADO  (JOIN a oficina para desnormalizar)
INSERT INTO dbo.STG_EMPLEADO (
    STG_ID_EMPLEADO, STG_NOMBRE_COMPLETO, STG_PUESTO,
    STG_OFI_CODIGO, STG_OFI_CIUDAD, STG_OFI_PAIS, STG_OFI_REGION,
    STG_FECHA_CARGA, STG_ORIGEN
)
SELECT
    e.ID_empleado,
    -- Consolidar nombre completo, eliminando apellido2 vacío:
    LTRIM(RTRIM(e.nombre + ' ' + e.apellido1
          + ISNULL(' ' + NULLIF(LTRIM(RTRIM(e.apellido2)),''), '''))),
    LTRIM(RTRIM(e.puesto)),
    -- Desnormalizar oficina:
```

```
    LTRIM(RTRIM(o.Descripcion)),
    UPPER(LTRIM(RTRIM(o.ciudad))),
    UPPER(LTRIM(RTRIM(o.pais))),
    NULLIF(LTRIM(RTRIM(o.region)), ''),
    GETDATE(), 'jardineria'
FROM jardineria.dbo.empleado e
LEFT JOIN jardineria.dbo.oficina o ON e.ID_oficina = o.ID_oficina;
PRINT '  Paso 1/5: STG_EMPLEADO — ' + CAST(@@ROWCOUNT AS VARCHAR) + ' filas.';

-- Paso 2 — STG_CLIENTE  (JOIN a empleado + oficina para desnormalizar
representante)
INSERT INTO dbo.STG_CLIENTE (
    STG_ID_CLIENTE, STG_NOMBRE_CLIENTE, STG_CIUDAD, STG_REGION,
    STG_PAIS, STG_LIMITE_CREDITO,
    STG_REP_NOMBRE, STG_REP_PUESTO, STG_REP_OFI_CIUDAD, STG_REP_OFI_PAIS,
    STG_FECHA_CARGA, STG_ORIGEN
)
SELECT
    c.ID_cliente,
    LTRIM(RTRIM(c.nombre_cliente)),
    UPPER(LTRIM(RTRIM(c.ciudad))),
    NULLIF(LTRIM(RTRIM(c.region)), ''),
    UPPER(NULLIF(LTRIM(RTRIM(c.pais)), '')),
    ISNULL(c.limite_credito, 0),
    -- Desnormalizar representante de ventas:
    LTRIM(RTRIM(e.nombre + ' ' + e.apellido1
          + ISNULL(' ' + NULLIF(LTRIM(RTRIM(e.apellido2)),''), '')),
    LTRIM(RTRIM(e.puesto)),
    -- Desnormalizar oficina del representante:
    UPPER(LTRIM(RTRIM(o.ciudad))),
    UPPER(LTRIM(RTRIM(o.pais))),
    GETDATE(), 'jardineria'
FROM jardineria.dbo.cliente c
LEFT JOIN jardineria.dbo.empleado e ON c.ID_empleado_rep_ventas = e.ID_empleado
LEFT JOIN jardineria.dbo.oficina  o ON e.ID_oficina = o.ID_oficina;
PRINT '  Paso 2/5: STG_CLIENTE — ' + CAST(@@ROWCOUNT AS VARCHAR) + ' filas.';

-- Paso 3 — STG_PRODUCTO  (JOIN a Categoria_producto para desnormalizar)

INSERT INTO dbo.STG_PRODUCTO (
    STG_ID_PRODUCTO, STG_CODIGO_PRODUCTO, STG_NOMBRE,
    STG_CATEGORIA, STG_PROVEEDOR, STG_CANTIDAD_STOCK,
```

```
    STG_PRECIO_VENTA, STG_PRECIO_PROVEEDOR,
    STG_FECHA_CARGA, STG_ORIGEN
)
SELECT
    p.ID_producto,
    UPPER(LTRIM(RTRIM(p.CodigoProducto))),
    LTRIM(RTRIM(p.nombre)),
    -- Desnormalizar nombre de categoría:
    LTRIM(RTRIM(cp.Desc_Categoria)),
    NULLIF(LTRIM(RTRIM(p.proveedor)), ''),
    ISNULL(p.cantidad_en_stock, 0),
    p.precio_venta,
    p.precio_proveedor,
    GETDATE(), 'jardineria'
FROM jardineria.dbo.producto p
LEFT JOIN jardineria.dbo.Categoria_producto cp ON p.Categoria =
cp.Id_Categoria;
PRINT '  Paso 3/5: STG_PRODUCTO — ' + CAST(@@ROWCOUNT AS VARCHAR) + ' filas.';

-- Paso 4 — STG_PEDIDO  (JOIN a cliente para desnormalizar datos geográficos)

INSERT INTO dbo.STG_PEDIDO (
    STG_ID_PEDIDO, STG_FECHA_PEDIDO, STG_FECHA_ESPERADA,
    STG_FECHA_ENTREGA, STG_ESTADO, STG_ID_CLIENTE,
    STG_CLI_NOMBRE, STG_CLI_CIUDAD, STG_CLI_PAIS,
    STG_FECHA_CARGA, STG_ORIGEN
)
SELECT
    p.ID_pedido,
    CAST(p.fecha_pedido   AS DATE),
    CAST(p.fecha_esperada AS DATE),
    CAST(p.fecha_entrega  AS DATE),         -- puede ser NULL
    UPPER(LTRIM(RTRIM(p.estado))),
    p.ID_cliente,
    -- Desnormalizar datos del cliente:
    LTRIM(RTRIM(c.nombre_cliente)),
    UPPER(LTRIM(RTRIM(c.ciudad))),
    UPPER(NULLIF(LTRIM(RTRIM(c.pais)), '')),
    GETDATE(), 'jardineria'
FROM jardineria.dbo.pedido p
LEFT JOIN jardineria.dbo.cliente c ON p.ID_cliente = c.ID_cliente;
PRINT '  Paso 4/5: STG_PEDIDO — ' + CAST(@@ROWCOUNT AS VARCHAR) + ' filas.';
```

```
-- Paso 5 — STG_DETALLE_PEDIDO  (JOIN a producto + categoría para
desnormalizar)

INSERT INTO dbo.STG_DETALLE_PEDIDO (
    STG_ID_PEDIDO, STG_ID_PRODUCTO, STG_CANTIDAD, STG_PRECIO_UNIDAD,
    STG_PRO_NOMBRE, STG_PRO_CATEGORIA,
    STG_FECHA_CARGA, STG_ORIGEN
)
SELECT
    dp.ID_pedido,
    dp.ID_producto,
    dp.cantidad,
    dp.precio_unidad,
    -- Desnormalizar nombre y categoría del producto:
    LTRIM(RTRIM(p.nombre)),
    LTRIM(RTRIM(cp.Desc_Categoria)),
    GETDATE(), 'jardineria'
FROM jardineria.dbo.detalle_pedido dp
LEFT JOIN jardineria.dbo.producto          p  ON dp.ID_producto = p.ID_producto
LEFT JOIN jardineria.dbo.Categoria_producto cp ON p.Categoria   =
cp.Id_Categoria
WHERE dp.cantidad > 0 AND dp.precio_unidad > 0;
PRINT '  Paso 5/5: STG_DETALLE_PEDIDO — ' + CAST(@@ROWCOUNT AS VARCHAR) + '
filas.';

-- Paso 6 — STG_PAGO  (JOIN a cliente para desnormalizar nombre)

INSERT INTO dbo.STG_PAGO (
    STG_ID_CLIENTE, STG_CLI_NOMBRE, STG_FORMA_PAGO,
    STG_ID_TRANSACCION, STG_FECHA_PAGO, STG_TOTAL,
    STG_FECHA_CARGA, STG_ORIGEN
)
SELECT
    pg.ID_cliente,
    -- Desnormalizar nombre del cliente:
    LTRIM(RTRIM(c.nombre_cliente)),
    LTRIM(RTRIM(pg.forma_pago)),
    LTRIM(RTRIM(pg.id_transaccion)),
    CAST(pg.fecha_pago AS DATE),
    pg.total,
    GETDATE(), 'jardineria'
```

```
FROM jardineria.dbo.pago pg
LEFT JOIN jardineria.dbo.cliente c ON pg.ID_cliente = c.ID_cliente
WHERE pg.total > 0;
PRINT '  STG_PAGO — ' + CAST(@@ROWCOUNT AS VARCHAR) + ' filas.';

PRINT '>>> Fase 3 completada: ETL ejecutado correctamente.';

-- validacion de datos
-- 4.1 Conteo de Registros — Fuente vs. Staging (CORREGIDO)
SELECT
    Tabla,
    MAX(CASE WHEN Origen = 'FUENTE'  THEN Total ELSE 0 END) AS Filas_Fuente,
    MAX(CASE WHEN Origen = 'STAGING' THEN Total ELSE 0 END) AS Filas_Staging,
    MAX(CASE WHEN Origen = 'FUENTE'  THEN Total ELSE 0 END)
  - MAX(CASE WHEN Origen = 'STAGING' THEN Total ELSE 0 END) AS Diferencia
FROM (
    SELECT 'empleado'           AS Tabla, 'FUENTE'  AS Origen, COUNT(*) AS
Total FROM jardineria.dbo.empleado            UNION ALL
    SELECT 'STG_EMPLEADO'       AS Tabla, 'STAGING' AS Origen, COUNT(*) AS
Total FROM jardineria_staging.dbo.STG_EMPLEADO UNION ALL
    SELECT 'cliente'            AS Tabla, 'FUENTE'  AS Origen, COUNT(*) AS
Total FROM jardineria.dbo.cliente              UNION ALL
    SELECT 'STG_CLIENTE'        AS Tabla, 'STAGING' AS Origen, COUNT(*) AS
Total FROM jardineria_staging.dbo.STG_CLIENTE  UNION ALL
    SELECT 'producto'           AS Tabla, 'FUENTE'  AS Origen, COUNT(*) AS
Total FROM jardineria.dbo.producto             UNION ALL
    SELECT 'STG_PRODUCTO'       AS Tabla, 'STAGING' AS Origen, COUNT(*) AS
Total FROM jardineria_staging.dbo.STG_PRODUCTO UNION ALL
    SELECT 'pedido'             AS Tabla, 'FUENTE'  AS Origen, COUNT(*) AS
Total FROM jardineria.dbo.pedido               UNION ALL
    SELECT 'STG_PEDIDO'         AS Tabla, 'STAGING' AS Origen, COUNT(*) AS
Total FROM jardineria_staging.dbo.STG_PEDIDO   UNION ALL
    SELECT 'detalle_pedido'     AS Tabla, 'FUENTE'  AS Origen, COUNT(*) AS
Total FROM jardineria.dbo.detalle_pedido               UNION ALL
    SELECT 'STG_DETALLE_PEDIDO' AS Tabla, 'STAGING' AS Origen, COUNT(*) AS
Total FROM jardineria_staging.dbo.STG_DETALLE_PEDIDO   UNION ALL
    SELECT 'pago'               AS Tabla, 'FUENTE'  AS Origen, COUNT(*) AS
Total FROM jardineria.dbo.pago                 UNION ALL
    SELECT 'STG_PAGO'           AS Tabla, 'STAGING' AS Origen, COUNT(*) AS
Total FROM jardineria_staging.dbo.STG_PAGO
) AS src
GROUP BY Tabla
```

```
ORDER BY Tabla;

SELECT 'STG_EMPLEADO — OFI_CIUDAD nula'       AS Problema, COUNT(*) AS
Registros FROM dbo.STG_EMPLEADO       WHERE STG_OFI_CIUDAD IS NULL
UNION ALL
SELECT 'STG_CLIENTE — REP_NOMBRE nulo',                    COUNT(*) FROM
dbo.STG_CLIENTE  WHERE STG_REP_NOMBRE IS NULL AND STG_LIMITE_CREDITO > 0
UNION ALL
SELECT 'STG_PRODUCTO — CATEGORIA nula',                    COUNT(*) FROM
dbo.STG_PRODUCTO WHERE STG_CATEGORIA IS NULL
UNION ALL
SELECT 'STG_PEDIDO — CLI_NOMBRE nulo',                     COUNT(*) FROM
dbo.STG_PEDIDO   WHERE STG_CLI_NOMBRE IS NULL
UNION ALL
SELECT 'STG_DETALLE — PRO_NOMBRE nulo',                    COUNT(*) FROM
dbo.STG_DETALLE_PEDIDO WHERE STG_PRO_NOMBRE IS NULL
UNION ALL
SELECT 'STG_PAGO — CLI_NOMBRE nulo',                       COUNT(*) FROM
dbo.STG_PAGO     WHERE STG_CLI_NOMBRE IS NULL;
-- Resultado esperado: 0 en todas las filas

SELECT 'FUENTE'  AS Origen, SUM(CAST(cantidad AS NUMERIC(15,2)) *
precio_unidad) AS Total_Ventas
FROM jardineria.dbo.detalle_pedido WHERE cantidad > 0 AND precio_unidad > 0
UNION ALL
SELECT 'STAGING', SUM(STG_IMPORTE_LINEA) FROM
jardineria_staging.dbo.STG_DETALLE_PEDIDO;

SELECT 'FUENTE'  AS Origen, SUM(total) AS Total_Pagos FROM jardineria.dbo.pago
WHERE total > 0
UNION ALL
SELECT 'STAGING', SUM(STG_TOTAL) FROM jardineria_staging.dbo.STG_PAGO;
-- Diferencia esperada: 0 en ambos casos

SELECT COUNT(*) AS Inconsistencias_Nombre_Cliente
FROM dbo.STG_PEDIDO    p
JOIN dbo.STG_CLIENTE   c ON p.STG_ID_CLIENTE = c.STG_ID_CLIENTE
WHERE p.STG_CLI_NOMBRE <> c.STG_NOMBRE_CLIENTE;

-- Verificar que categoría en STG_DETALLE coincide con STG_PRODUCTO
SELECT COUNT(*) AS Inconsistencias_Categoria_Producto
FROM dbo.STG_DETALLE_PEDIDO d
```

```
JOIN dbo.STG_PRODUCTO       p ON d.STG_ID_PRODUCTO = p.STG_ID_PRODUCTO
WHERE d.STG_PRO_CATEGORIA <> p.STG_CATEGORIA;
-- Resultado esperado: 0 en ambas consultas

-- Consultas analiticas de negocio
--¿Qué oficina genera más ventas?
SELECT
    c.STG_REP_OFI_CIUDAD                            AS Ciudad_Oficina,
    c.STG_REP_OFI_PAIS                              AS Pais_Oficina,
    COUNT(DISTINCT p.STG_ID_PEDIDO)                 AS Total_Pedidos,
    COUNT(DISTINCT p.STG_ID_CLIENTE)                AS Clientes_Atendidos,
    SUM(d.STG_IMPORTE_LINEA)                        AS Importe_Total_Ventas,
    RANK() OVER (ORDER BY SUM(d.STG_IMPORTE_LINEA) DESC) AS Ranking
FROM      dbo.STG_DETALLE_PEDIDO d
JOIN      dbo.STG_PEDIDO         p ON d.STG_ID_PEDIDO  = p.STG_ID_PEDIDO
JOIN      dbo.STG_CLIENTE        c ON p.STG_ID_CLIENTE = c.STG_ID_CLIENTE
GROUP BY  c.STG_REP_OFI_CIUDAD, c.STG_REP_OFI_PAIS
ORDER BY  Importe_Total_Ventas DESC;

--¿Qué categoría de producto prefiere cada cliente?
WITH RankedCategorias AS (
    SELECT
        p.STG_CLI_NOMBRE                            AS Cliente,
        d.STG_PRO_CATEGORIA                         AS Categoria,
        SUM(d.STG_CANTIDAD)                         AS Unidades,
        SUM(d.STG_IMPORTE_LINEA)                    AS Importe,
        ROW_NUMBER() OVER (
            PARTITION BY p.STG_ID_CLIENTE
            ORDER BY SUM(d.STG_IMPORTE_LINEA) DESC
        )                                           AS Rn
    FROM  dbo.STG_DETALLE_PEDIDO d
    JOIN  dbo.STG_PEDIDO         p ON d.STG_ID_PEDIDO = p.STG_ID_PEDIDO
    GROUP BY p.STG_ID_CLIENTE, p.STG_CLI_NOMBRE, d.STG_PRO_CATEGORIA
)
SELECT Cliente, Categoria AS Categoria_Favorita, Unidades, Importe
FROM   RankedCategorias
WHERE  Rn = 1
ORDER BY Importe DESC;

--Ranking global de categorías de producto
SELECT
    STG_PRO_CATEGORIA                               AS Categoria,
```

```
    COUNT(DISTINCT STG_ID_PEDIDO)                   AS Pedidos,
    SUM(STG_CANTIDAD)                               AS Unidades_Totales,
    SUM(STG_IMPORTE_LINEA)                          AS Importe_Total,
    ROUND(100.0 * SUM(STG_IMPORTE_LINEA)
        / SUM(SUM(STG_IMPORTE_LINEA)) OVER (), 2)  AS Pct_Sobre_Total
FROM  dbo.STG_DETALLE_PEDIDO
GROUP BY STG_PRO_CATEGORIA
ORDER BY Importe_Total DESC;

--Análisis de rentabilidad de productos — Top 10 por margen
SELECT TOP 10
    pro.STG_NOMBRE                                  AS Producto,
    pro.STG_CATEGORIA                               AS Categoria,
    pro.STG_PRECIO_VENTA                            AS Precio_Venta,
    pro.STG_PRECIO_PROVEEDOR                        AS Precio_Costo,
    pro.STG_MARGEN_BRUTO                            AS Margen_Bruto,
    ROUND(100.0 * pro.STG_MARGEN_BRUTO
        / pro.STG_PRECIO_VENTA, 2)                  AS Pct_Margen,
    SUM(det.STG_CANTIDAD)                           AS Unidades_Vendidas,
    SUM(det.STG_IMPORTE_LINEA)                      AS Ingreso_Total
FROM      dbo.STG_PRODUCTO        pro
LEFT JOIN dbo.STG_DETALLE_PEDIDO  det ON pro.STG_ID_PRODUCTO =
det.STG_ID_PRODUCTO
GROUP BY  pro.STG_NOMBRE, pro.STG_CATEGORIA,
          pro.STG_PRECIO_VENTA, pro.STG_PRECIO_PROVEEDOR, pro.STG_MARGEN_BRUTO
ORDER BY  pro.STG_MARGEN_BRUTO DESC;

--Análisis temporal de ventas por año y mes
SELECT
    p.STG_ANIO_PEDIDO                               AS Anio,
    p.STG_MES_PEDIDO                                AS Mes,
    COUNT(DISTINCT p.STG_ID_PEDIDO)                 AS Pedidos,
    SUM(d.STG_IMPORTE_LINEA)                        AS Ventas_Mes,
    SUM(SUM(d.STG_IMPORTE_LINEA)) OVER (
        PARTITION BY p.STG_ANIO_PEDIDO
        ORDER BY p.STG_MES_PEDIDO
        ROWS UNBOUNDED PRECEDING
    )                                               AS Ventas_Acumuladas_Anio
FROM  dbo.STG_DETALLE_PEDIDO d
JOIN  dbo.STG_PEDIDO         p ON d.STG_ID_PEDIDO = p.STG_ID_PEDIDO
GROUP BY p.STG_ANIO_PEDIDO, p.STG_MES_PEDIDO
ORDER BY p.STG_ANIO_PEDIDO, p.STG_MES_PEDIDO;
```

```
--Desempeño por representante de ventas
SELECT
    c.STG_REP_NOMBRE                                AS Representante,
    c.STG_REP_OFI_CIUDAD                            AS Ciudad_Oficina,
    COUNT(DISTINCT p.STG_ID_PEDIDO)                 AS Pedidos_Gestionados,
    COUNT(DISTINCT p.STG_ID_CLIENTE)                AS Clientes_Activos,
    SUM(d.STG_IMPORTE_LINEA)                        AS Ventas_Totales,
    RANK() OVER (ORDER BY SUM(d.STG_IMPORTE_LINEA) DESC) AS Ranking_Ventas
FROM      dbo.STG_DETALLE_PEDIDO d
JOIN      dbo.STG_PEDIDO         p ON d.STG_ID_PEDIDO  = p.STG_ID_PEDIDO
JOIN      dbo.STG_CLIENTE        c ON p.STG_ID_CLIENTE = c.STG_ID_CLIENTE
WHERE     c.STG_REP_NOMBRE IS NOT NULL
GROUP BY  c.STG_REP_NOMBRE, c.STG_REP_OFI_CIUDAD
ORDER BY  Ventas_Totales DESC;
```

Anexo C. Backup de la Base de Datos Jardinería (Origen)

El archivo de respaldo completo de la base de datos Jardinería, generado antes de ejecutar el proceso ETL, corresponde al archivo: https://drive.google.com/file/d/1H5BEP9tTOGD6ndPQw0CSuPwj-ocZZrNp/view?usp=drive_link. Este archivo debe restaurarse siguiendo el procedimiento estándar de SQL Server con la sentencia RESTORE DATABASE.

Comando de backup utilizado:

```
DECLARE @backupPath NVARCHAR(500)
SET @backupPath =
N'/var/opt/mssql/backups/jardineria_FULL_20260327.bak'
```

```
BACKUP DATABASE [jardineria]

TO DISK = @backupPath

WITH FORMAT,

     MEDIANAME = 'JardineriaBackup',

     NAME = 'Backup Completo - jardineria',

     COMPRESSION,

     CHECKSUM,

     STATS = 10;
```

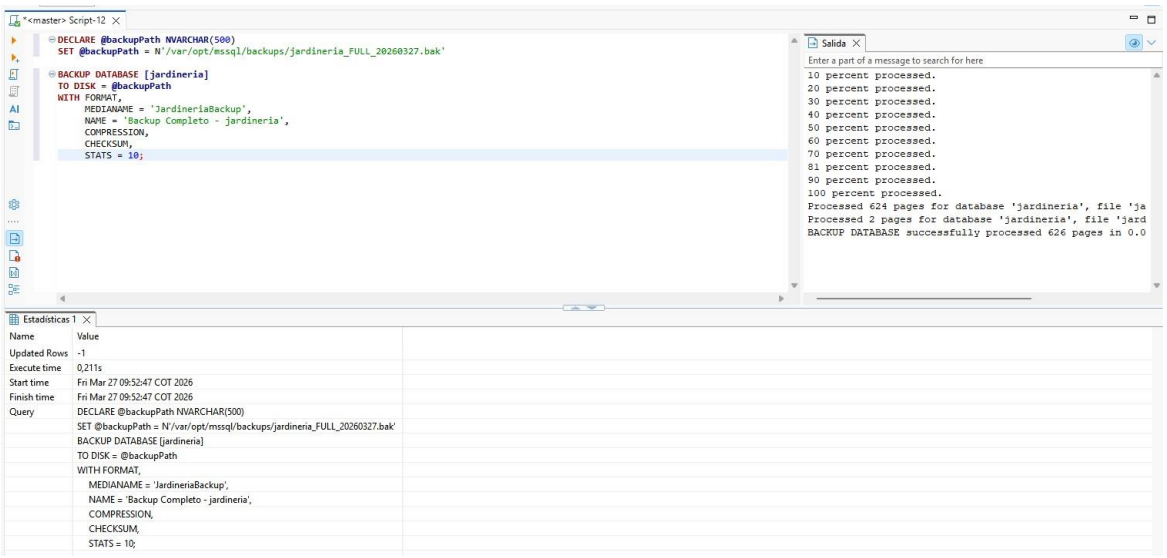

Figura 17

Ejecución del backup de la base de datos Jardinería

Anexo D. Backup de la Base de Datos Staging

El archivo de respaldo de la base de datos jardineria_staging, generado después de validar exitosamente la carga ETL, corresponde al archivo:

https://drive.google.com/file/d/1hxeMVo3UgvXof_LTAVACsuknCJWCWO-I/view?usp=drive_link.

El procedimiento automatizado de backup se implementó mediante el procedimiento almacenado dbo.usp_Backup_Staging.

Procedimiento de backup automático:

```
DECLARE @ruta NVARCHAR(500);
DECLARE @fecha VARCHAR(8) = CONVERT(VARCHAR, GETDATE(), 112);
SET @ruta = N'/var/opt/mssql/backups/jardineria_staging_FULL_' + @fecha
+ '.bak';

BACKUP DATABASE [jardineria_staging]
TO DISK = @ruta
WITH FORMAT,
     COMPRESSION,
     CHECKSUM,
     STATS = 10;
```

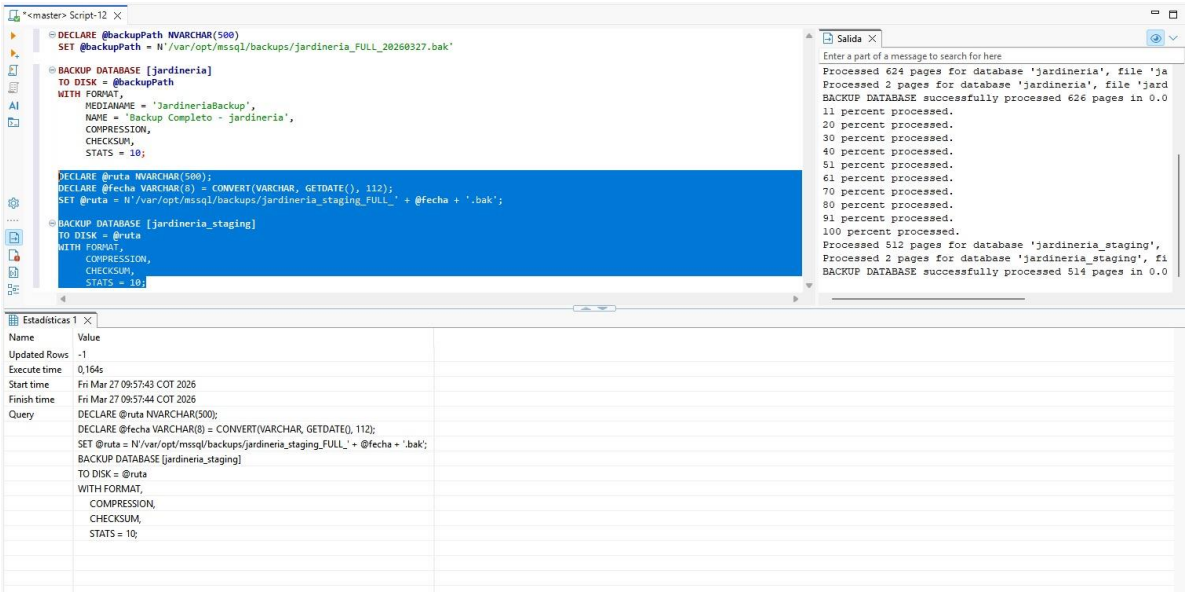

Figura 18

Ejecución del backup de la base de datos jardineria_staging

Anexo E. Resumen de la Auditoría de Campos

La siguiente tabla resume los resultados de la auditoría de campos realizada sobre las ocho tablas de la base de datos Jardinería:

Tabla Origen	Conserva	Descarta	Desnorm.	Calculado
oficina	0	5	4	0
Categoria_producto	0	4	1	0
empleado	2	4	4	0
cliente	5	6	4	0
producto	5	3	1	1
pedido	5	1	3	3
detalle_pedido	4	1	2	1
pago	4	0	1	2
TOTAL	**22**	**25**	**11**	**7**

Figura 19

Resumen de la auditoría de campos de la base de datos Jardinería

Referencias

Alzate Montoya, L. F. (2021). Bases de datos: fundamentos, diseño y administración (2.ª ed.). Editorial Universidad de Antioquia.

Inmon, W. H. (2005). Building the data warehouse (4th ed.). Wiley.

Kimball, R., & Ross, M. (2013). The data warehouse toolkit: The definitive guide to dimensional modeling (3rd ed.). Wiley.

Microsoft Corporation. (2024). SQL Server 2016: Transact-SQL reference. https://learn.microsoft.com/en-us/sql/t-sql/

Microsoft Corporation. (2024). BACKUP (Transact-SQL). Microsoft Learn. https://learn.microsoft.com/en-us/sql/t-sql/statements/backup-transact-sql

Microsoft Corporation. (2024). Computed columns. Microsoft Learn. https://learn.microsoft.com/en-us/sql/relational-databases/tables/specify-computed-columns-in-a-table

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database system concepts (7th ed.). McGraw-Hill Education.

Teorey, T. J., Lightstone, S. S., Nadeau, T., & Jagadish, H. V. (2011). Database modeling and design: Logical design (5th ed.). Morgan Kaufmann.